

- **Task A -Data Quality & Preprocessing:**

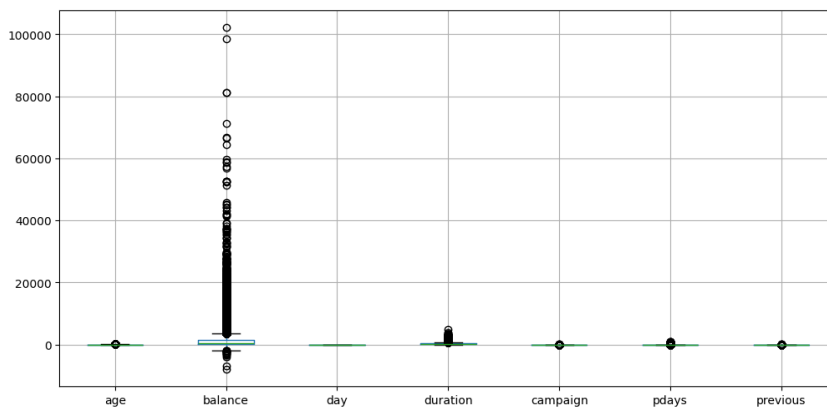
- 1) **Identify missing values, duplicates, and noisy/outlier data:**

#	Column	Non-Null	Count	Dtype
---	-----	-----	-----	-----
0	age	45211	non-null	int64
1	job	45211	non-null	str
2	marital	45211	non-null	str
3	education	45211	non-null	str
4	default	45211	non-null	str
5	balance	45211	non-null	int64
6	housing	45211	non-null	str
7	loan	45211	non-null	str
8	contact	45211	non-null	str
9	day	45211	non-null	int64
10	month	45211	non-null	str
11	duration	45211	non-null	int64
12	campaign	45211	non-null	int64
13	pdays	45211	non-null	int64
14	previous	45211	non-null	int64
15	poutcome	45211	non-null	str
16	y	45211	non-null	str

Duplicates datapoints:

```
df.duplicated().sum()
✓ 0.3s
np.int64(0)
```

noisy/outlier data:



- 2) **Explain the types of attributes:**

```
Numerical features: Index(['age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous'], dtype='str')
Categorical features: Index(['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact',
                             'month', 'poutcome'],
                             dtype='str')
```

3) Apply preprocessing:

I have used a power transformer in which I apply OneHotEncoding on categorical features, and a standard scaler on numerical features.

I applied a Label encoder separately on the target column.

4) Split the dataset into Train/Test:

```
x_train, x_test, y_train, y_test = train_test_split(df.drop("y", axis = 1), df['y'], test_size = 0.2, random_state = 42)
✓ 0.1s
```

● Task B -Model Training + Evaluation:

1) Logistic Regression (Baseline model):

Classification matrix –

	precision	recall	f1-score	support
0	0.92	0.98	0.94	7952
1	0.65	0.34	0.45	1091
accuracy			0.90	9043
macro avg	0.78	0.66	0.70	9043
weighted avg	0.88	0.90	0.88	9043

● Task C -Decision Tree Learning + Interpretation:

Baseline model (without tuning)--

	precision	recall	f1-score	support
0	0.93	0.93	0.93	7952
1	0.48	0.49	0.48	1091
accuracy			0.87	9043
macro avg	0.70	0.71	0.71	9043
weighted avg	0.88	0.87	0.87	9043

With tuning –

	precision	recall	f1-score	support
0	0.91	0.97	0.94	7952
1	0.65	0.33	0.44	1091
accuracy			0.90	9043
macro avg	0.78	0.65	0.69	9043
weighted avg	0.88	0.90	0.88	9043

Both precision and accuracy have improved by tuning, but recall has reduced from 49 to 33.

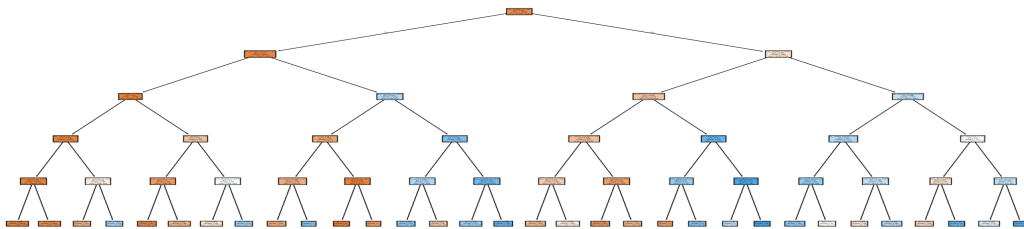
Feature importance –

Features that have a greater say in prediction

feature	importance
num__duration	0.569681
cat__poutcome_success	0.299584
num__age	0.049397
cat__month_mar	0.039760
cat__contact_unknown	0.017580
cat__month_may	0.006090
cat__housing_no	0.004612
num__balance	0.002265
num__day	0.002221
cat__marital_married	0.001792

● Task D -Rule-Based Classification:

Rule-Based Classification using a rule extraction approach from a decision tree



The decision tree is trained with max depth = 5, min samples split = 10, and min samples leaf = 5.

5 rules extracted from the decision tree —

1) Rule 1:

if duration <= 0.99 and poutcome_success <= 0.50 and age <= 1.84
then customer will not subscribe term deposit

2) Rule 2:

if duration <= 0.99 and poutcome_success <= 0.50 and age <= 1.84 and
month = march and duraion > -0.40 then customer will subscribe the
term deposit

3) Rule 3:

if duration <= 0.99 and poutcome_success > 0.50 and age > 1.84 and
duration <= -0.46 and month = may then customer will not subscribe

4) Rule 4:

duration > 0.99 and duration <= 2.25 and poutcome_success <= 0.50
then customer will not subscribe

5) Rule 5:

duration > 0.99 and duration > 2.25 and contact_unknown <= 0.50
and marital_status = unmarried and month_nov <= 0.50 then customer
will subscribe the term deposit

Explain how rules affect interpretability vs performance –

1) Interpretability:

- (a) Rules are easy to interpret
- (b) They can be used by businesses easily without the help of experts

2) Performance:

- (a) Rules are generally used for simplification and are less capable of learning complex patterns.

● Task E -kNN (Lazy Learning):

```
=====
value of k= 1
accuracy score: 0.8814552692690479
value of k= 3
accuracy score: 0.8925135463894726
value of k= 5
accuracy score: 0.8984850160345018
value of k= 7
accuracy score: 0.8979321021784806
```

```
value of k= 9
accuracy score: 0.8995908437465443
=====
```

```
metrics = ['euclidean', 'manhattan']

for (variable) knn: KNeighborsClassifier
    knn = KNeighborsClassifier(n_neighbors=5, metric=m)
    knn.fit(x_train_encoded, y_train_encoded)

    y_pred = knn.predict(x_test_encoded)

    print(f"\nMetric = {m}")
    print("Accuracy:", accuracy_score(y_test_encoded, y_pred))
```

✓ 25.1s

Metric = euclidean
Accuracy: 0.8984850160345018

Metric = manhattan
Accuracy: 0.8991485126617274

Why scaling is critical for kNN?

knn is distance based machine learning model. Whenever we give test data to the model, it first computes the distance with all datapoints then it identify k nearest neighbour as per the distance, and predict value by counting average of selected k values. In this process, distance becomes crucial component of prediction. To calculate the distance we use Euclidean and manhattan distance formulas, and if we give either of the values in a higher range, there are very high chances that the higher value dominates the result, suppressing the significance of the other value. This skewedness can cause incorrect prediction.

How does k affect bias/variance?

Low value of k: It led to high variance and low bias. It happens because the model finds the precise value of the train datapoint.

High value of k: It led to low variance and high bias. In such cases, the model does not find the precise value to predict for the test data, rather it calculates the average of k nearest neighbor datapoints.

- **Task F - Ensemble Learning:**

For random forest –

Random Forest Accuracy: 0.9034612407386929				
	precision	recall	f1-score	support
0	0.92	0.97	0.95	7952
1	0.66	0.41	0.51	1091
accuracy			0.90	9043
macro avg	0.79	0.69	0.73	9043
weighted avg	0.89	0.90	0.89	9043

For adaboosting –

AdaBoost Accuracy: 0.8938405396439235				
	precision	recall	f1-score	support
0	0.92	0.97	0.95	7952
1	0.66	0.42	0.51	1091
accuracy			0.90	9043
macro avg	0.79	0.69	0.73	9043
weighted avg	0.89	0.90	0.89	9043

- **Task G -Final Comparison + Recommendation:**

	Model	Accuracy	F1 Score	Notes
0	Logistic Regression	0.898706	0.448857	Baseline model
1	Decision Tree	0.873272	0.483318	prone to overfitting
2	Tuned Decision Tree	0.897490	0.439201	prone to overfitting
3	kNN	0.899591	0.449029	Sensitive to k
4	Random Forest	0.903461	0.507614	performance
5	Gradient Boosting	0.893841	0.413203	accuracy

1. Which model performed best and why?

It is a random forest. It uses parallel architecture with diverse decision trees, each having high variance. The combined power of diverse, parallel decision trees makes it the best performer.

2. Which model is most interpretable and why?

It is a rule-based classification. The rules derived from the decision tree are highly interpretable and can be understood even by the manager teams in day-to-day work.

3. If this model were deployed in real life, what risks exist?

Rule-based classification is very simple and can not work well on complex patterns in the dataset. Using it can cause incorrect predictions.